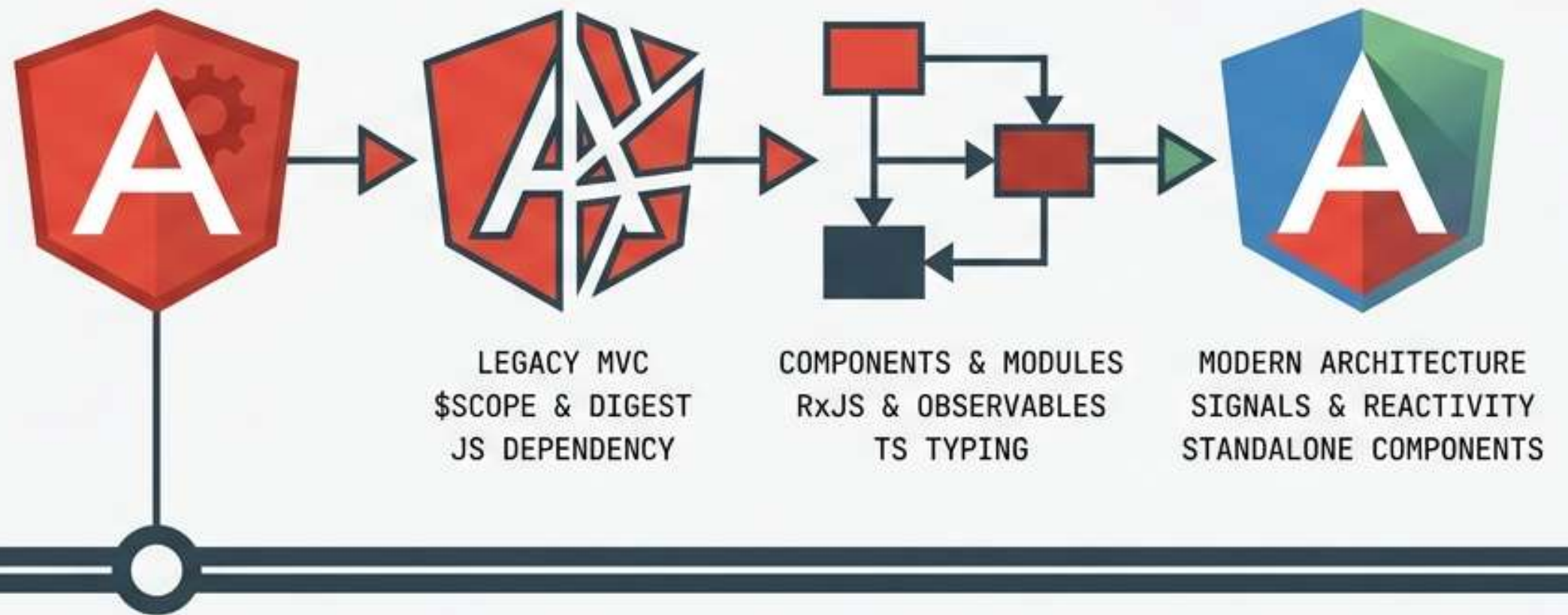


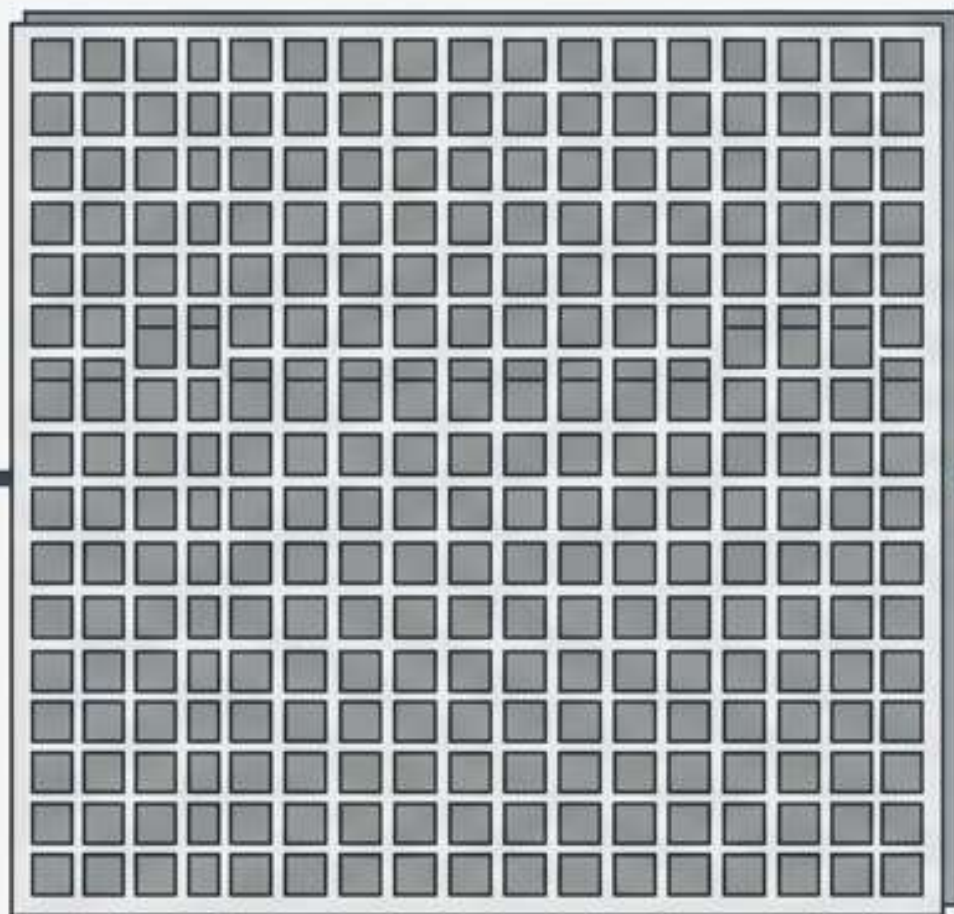
Эволюция Angular: От GetAngular до сверхточной реактивности

Исторический анализ архитектуры, цена технического долга и стратегии миграции легаси-систем в 2026 году.

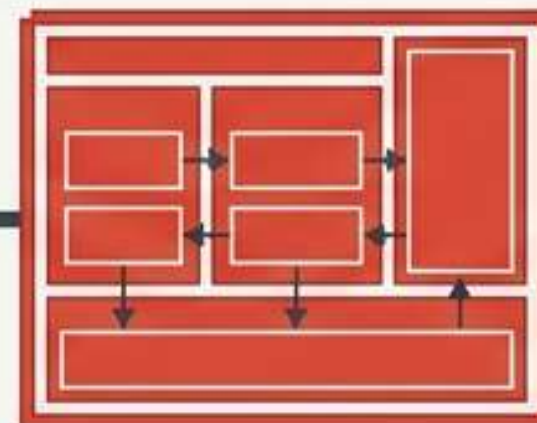


Чудо сокращения кода: 17 000 → 1 500 строк

IMPERATIVE CODE



DECLARATIVE CODE



2009

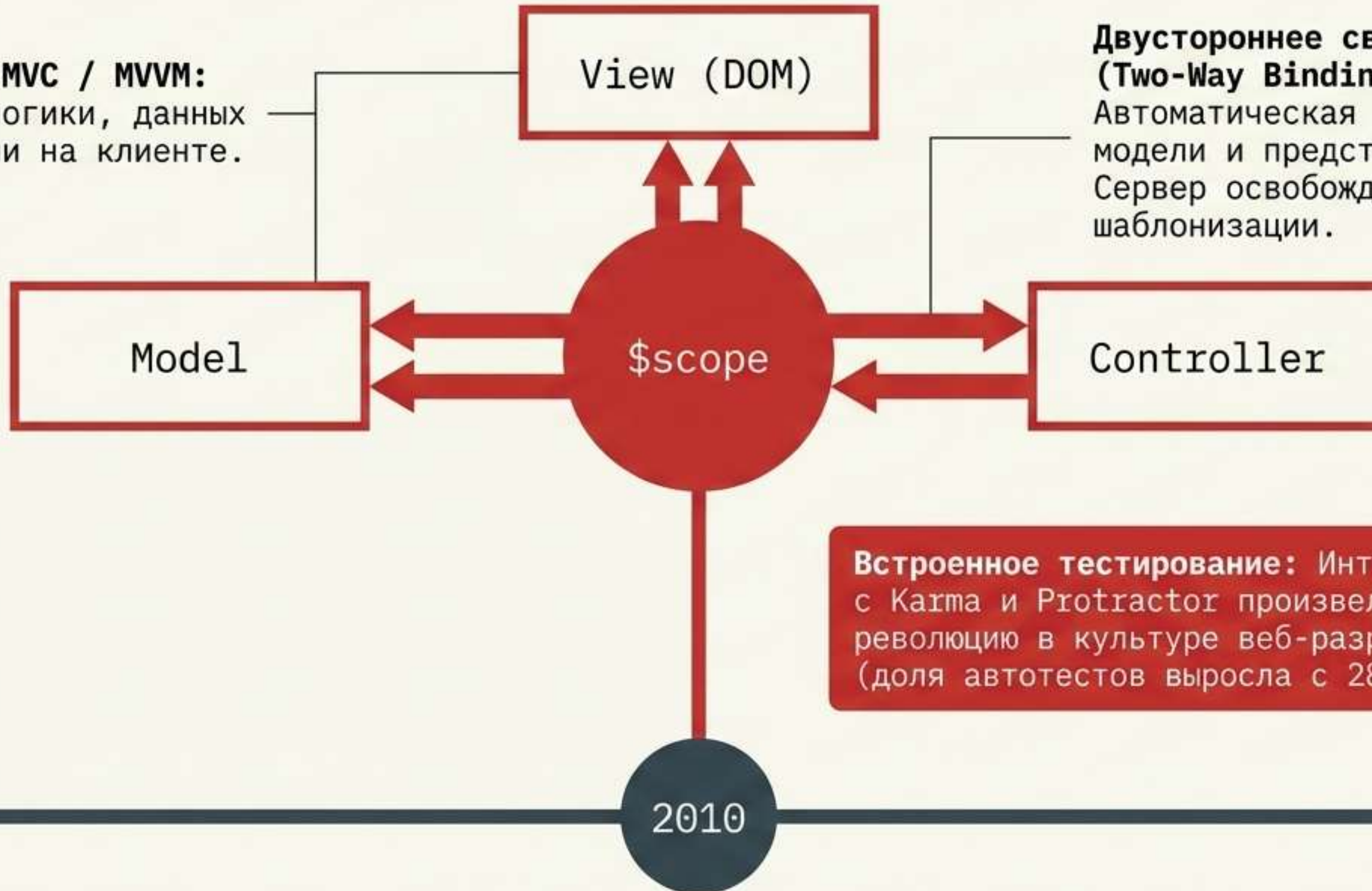
6 месяцев разработки **Google Feedback** на стандартном **JavaScript** = 17 000 строк кода. Колоссальная сложность тестирования.

Пари Мишко Хевери (Miško Hevery): проект полностью переписан на **GetAngular** за 3 недели. Код сокращен до 1 500 строк.

Триумф декларативного программирования над императивными манипуляциями DOM.

Парадигма AngularJS и эпоха SPA

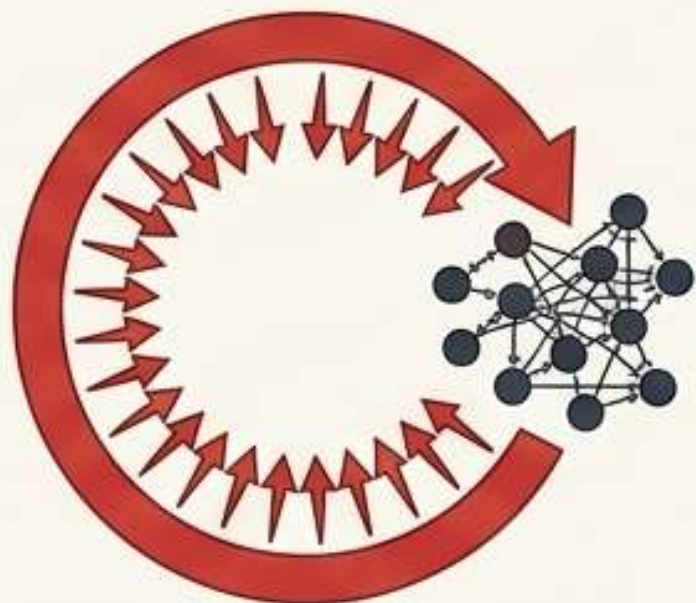
Архитектура MVC / MVVM:
Разделение логики, данных
и презентации на клиенте.



Двустороннее связывание (Two-Way Binding):
Автоматическая синхронизация
модели и представления.
Сервер освобожден от
шаблонизации.

Встроенное тестирование: Интеграция
с Karma и Protractor произвела
революцию в культуре веб-разработки
(доля автотестов выросла с 28% до 67%).

Великий раскол и кризис производительности



Цикл \$digest:
Метод «грязной проверки»
Сканирует все отслеживаемые переменные при каждом изменении. При масштабировании производительность лавинообразно падает.

Анонс Angular 2.0:
Представлен на NG-Conf 2014 без обратной совместимости.

Технологический
разрыв

2014-2016

Последствия: Разрыв технологических фундаментов потребовал полной перезаписи кода, подрвал доверие и открыл дорогу конкурентам (React, Vue).

Архитектурный разрыв: AngularJS vs Modern Angular

AngularJS

Modern Angular

Язык	JavaScript (динамическая типизация)	TypeScript (строгая типизация, интерфейсы, декораторы)
Архитектура	Контроллеры и \$scope (MVC)	Иерархия независимых компонентов
Обнаружение изменений	Цикл \$digest (полное сканирование)	Zone.js / Сигналы (однонаправленный поток данных)
Модульность	Кастомная система модулей	ES6 Модули и Standalone-компоненты

Стабилизация ядра: Монорепозиторий и SemVer



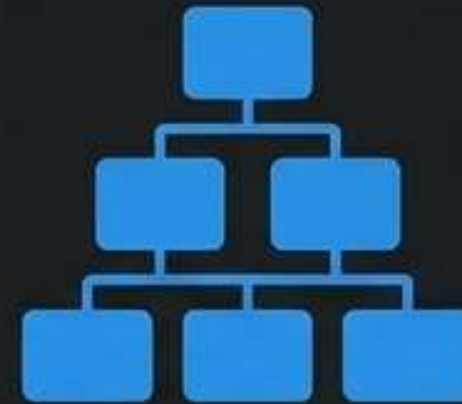
Преодоление «первородного греха» NgModules: Standalone API в деталях



Проблема NgModules

Внедрение NgModule в 2016 году было вынужденной мерой для AOT-компилятора.

Итог: высокий порог входа, избыточный шаблонный код (boilerplate) и сложная архитектура.



Решение: Standalone API

- **v14**: Режим Developer Preview.
- **v15**: Стабильные API без единого NgModule.
- **v19**: Standalone-архитектура включена по умолчанию.

Снижение барьера входа и упрощение структуры.

Сравнение методов создания компонентов

NgModule

Старый способ с NgModule

```
@NgModule({
  declarations: [MyComponent],
  imports: [CommonModule],
  exports: [MyComponent]
})
export class MyModule {}
```

Standalone

Новый способ Standalone

```
@Component({
  selector: 'my-comp',
  standalone: true,
  imports: [CommonModule]
})
export class MyComponent {}
```

Что сподвигло на изменения?

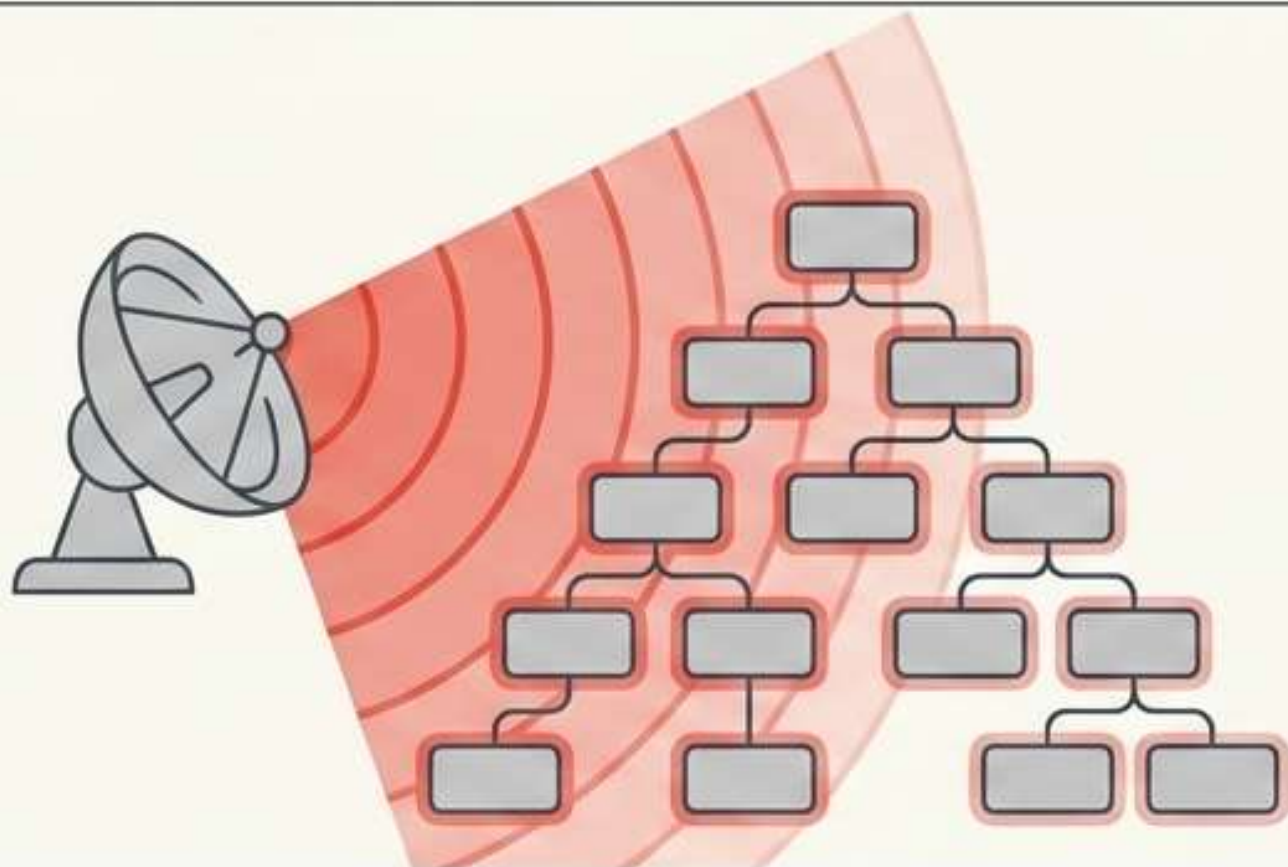
- Сложность понимания NgModules для новичков.
- Громоздкий код и зависимости.
- Трудности с tree-shaking.

Влияние других фреймворков

- **React**: Компонентно-ориентированный подход без модулей.
- **Vue**: Упрощенная система компонентов.
- Стремление к современным стандартам и упрощению разработки.

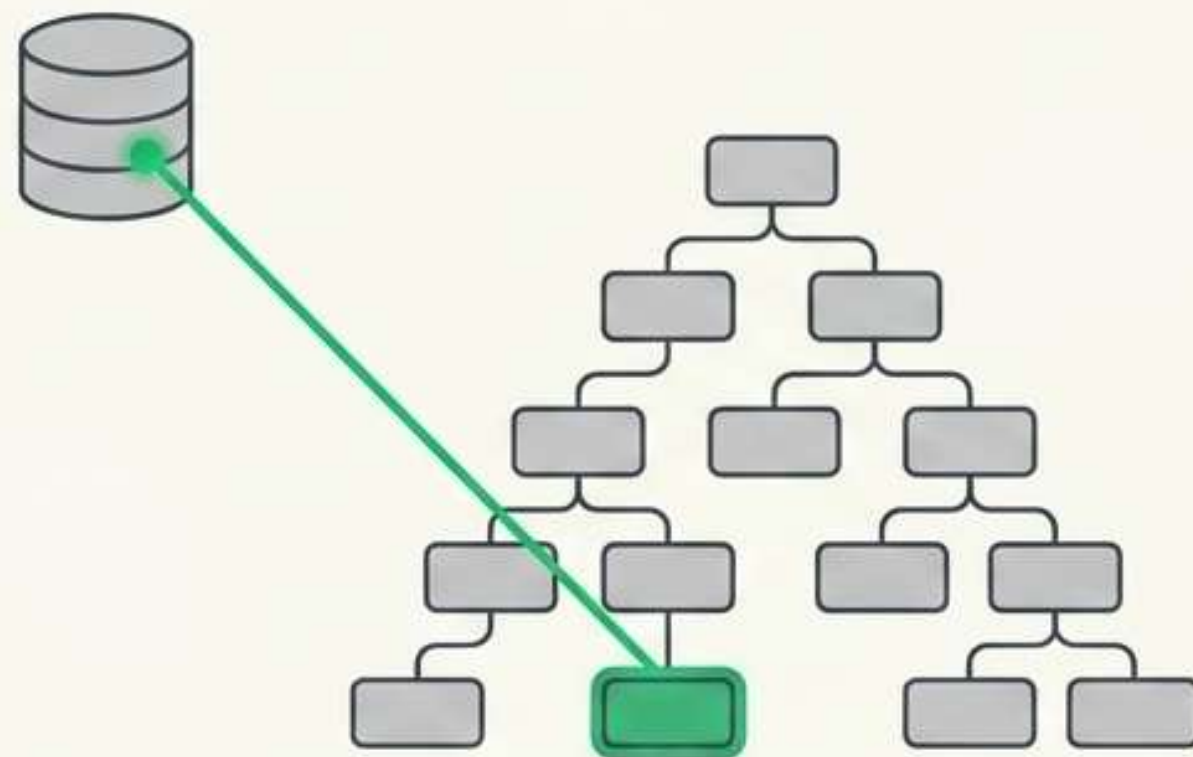
Революция реактивности: Внедрение Сигналов (Signals)

Зональное сканирование



Зональное сканирование: Zone.js отслеживает асинхронные операции, но вынужден проверять всё дерево компонентов сверху вниз при каждом событии.

Сверхточная реактивность



Сверхточная реактивность: Сигналы (Signals) обеспечивают мелкозернистую (fine-grained) реактивность. Фреймворк точно знает, какие данные изменились, и обновляет только один конкретный узел DOM.

v16+

Эра Zoneless: Максимальная производительность



Ускорение до 30-40%

времени начального рендеринга (Initial Render).

Снижение на 50%

ненужных повторных рендеров.

Отказ от тяжелой библиотеки Zone.js
(экспериментально в v18, стандарт в v21/v22).

- Существенное уменьшение размера бандла.
- Чистые и понятные трассировки стека (Stack Traces) для отладки.
- Прямое ручное управление циклом обновления UI.

Современный синтаксис шаблонов и Angular ARIA



- `div *ngIf="condition"`

+ `@if (condition) { ... }`

v17-v22

Встроенный декларативный синтаксис
(`@if`, `@for`, `@switch`).

Улучшает читаемость, гарантирует строгую проверку типов при компиляции и требует параметр `track` для циклов (исключая ошибки производительности).

Новые операторы в v20 (`**`, `in`).

Релиз в v21/v22. Набор функциональных директив (`Combobox`, `Accordion`, `Tree`), реализующих сложные паттерны **WAI-ARIA**.

Идеально для построения корпоративных дизайн-систем с высокой доступностью, оставляя полный контроль над визуалом.

Реальность 2026 года: Кризис End-of-Life (EOL)



**Поддержка AngularJS официально
прекращена 31 декабря 2021 года (EOL).**



Уязвимости

Навсегда неисправленные **CVE**-
CVE-уязвимости. Любая новая
угроза остается **открытой**.



Комплаенс

Прямое нарушение жестких
отраслевых стандартов
безопасности (HIPAA, PCI-DSS, SOC
2). Высокий риск провала аудита.



Кадровый голод

Разработчики избегают
устаревших технологий. **Острый
дефицит специалистов** для
поддержки легаси-систем.

Тот Angular, с которым вы спорили в 2021 году, больше не существует. Фреймворк потратил 3 года на удаление бойлерплейта.

Устаревшие паттерны



- @NgModule для всего
- Декораторы @Input() и сеттеры
- Классовые Guard'ы и ActivatedRoute
- Глобальная проверка zone.js

Новая парадигма



- **Standalone by default (v19+)**: Полная изоляция компонентов.
- **Signals API**: Реактивное состояние, input(), computed().
- **Zoneless & OnPush**: Точечный рендеринг, отказ от zone.js.
- **Functional API**: Роутинг, DI и гарды через чистые функции.

Анатомия Standalone-компонента

Больше не требуется включение в массив `declarations` какого-либо модуля.

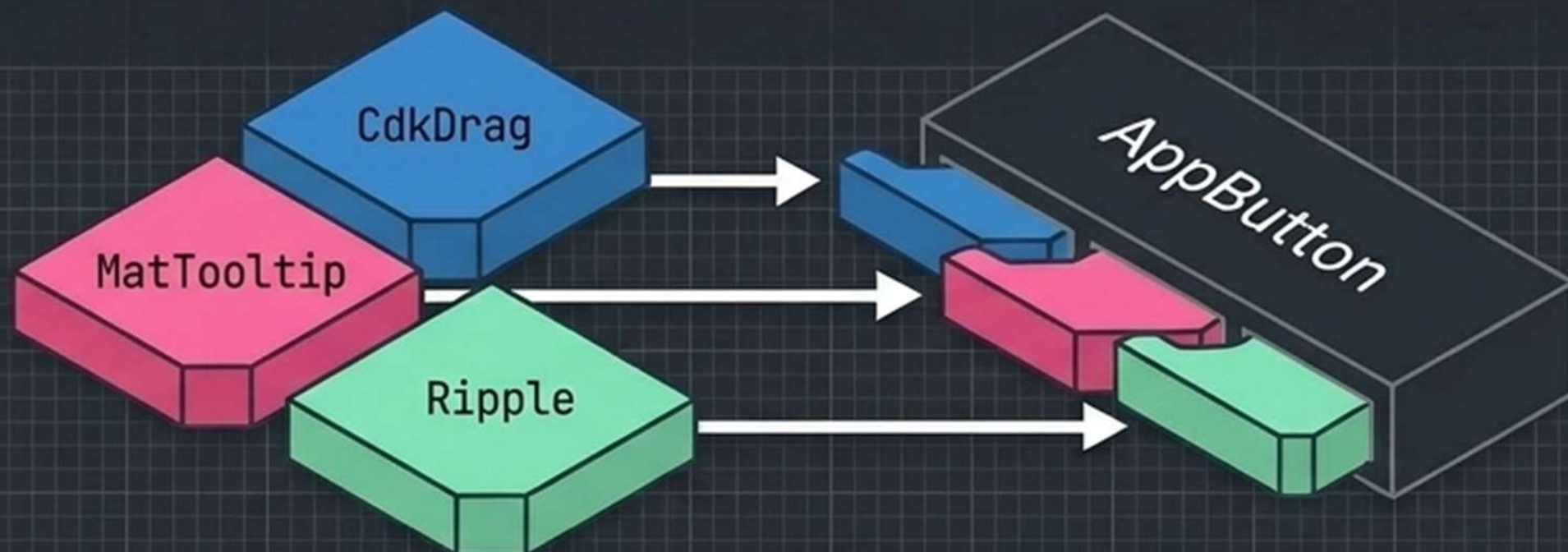
```
@Component({  
  selector: 'user-card',  
  standalone: true,  
  imports: [AvatarComponent, DatePipe],  
  template: ...  
})  
export class UserCardComponent {}
```

Идеальная база для Tree-Shaking. В продакшен-бандл попадает только то, что явно импортировано.

Прямое декларирование зависимостей (директивы, пайпы, компоненты). Формирует плоский и предсказуемый граф.

Композиция поведений: Directive Composition API

Решение проблемы множественного наследования. Свойство `hostDirectives` позволяет применять логику нескольких Standalone-директив к одному элементу без изменения HTML-шаблона.



```
@Component({
  selector: 'app-button',
  standalone: true,
  hostDirectives: [
    { directive: MatTooltip, inputs: ['matTooltip: tooltip'] },
    CdkDrag
  ]
})
```

[Переиспользование логики]

[Строгая типизация inputs/outputs]

[Отсутствие оберток в DOM]

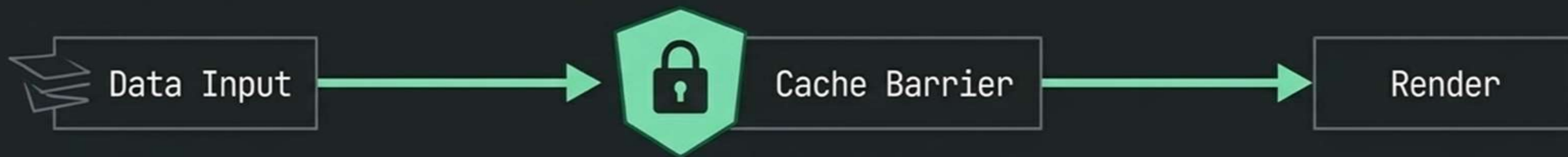
Матрица архитектурного выбора: Directives vs Pipes

Критерий	Directives	Pipes
Назначение	Изменение структуры и поведения DOM	Трансформация данных для отображения
Синтаксис	Атрибуты или структурные маркеры (<code>*ngIf</code> , <code>[ngClass]</code>)	Оператор конвейера (<code>{{ val format }}</code>)
Мутация данных	Могут изменять исходное состояние и слушать события 	Никогда не меняют исходные данные (Чистые функции) 
Пример	Фокус инпута, Drag-n-Drop	Форматирование дат, валют, локализация

Используйте директивы для управления поведением DOM. Используйте пайпы ИСКЛЮЧИТЕЛЬНО для визуального форматирования.

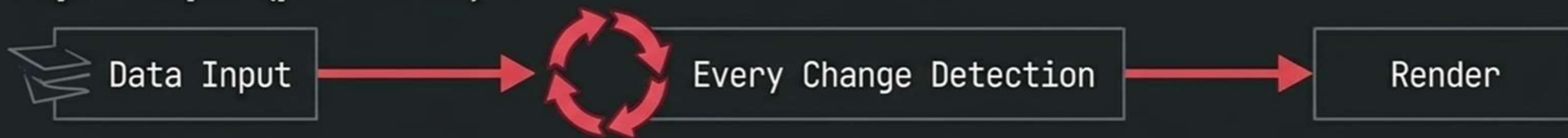
Производительность: Чистые и Нечистые Пайпы

Pure Pipes (По умолчанию - pure: true)



- Выполняются только при изменении примитива или ссылки на объект (Array/Object).
- Результат кэшируется. Идеально для производительности.

Impure Pipes (pure: false)



- Выполняются на каждый цикл Change Detection (сотни раз в секунду).
- Антипаттерн: Использование нечистых пайпов для фильтрации или сортировки массивов в шаблоне.

Best Practice

Вместо Impure Pipes для тяжелых вычислений выносите логику в сервисы или используйте реактивные вычисления `computed()` из Signals API.

Эволюция DI: Переход на функцию inject()

- **Устранение Boilerplate:** Больше не нужно раздувать конструкторы и передавать десятки параметров через `super()` при наследовании классов.
- **Функциональная композиция:** `inject()` позволяет писать переиспользуемые DI-функции вне контекста классов.
- **Улучшенный вывод типов:** TypeScript автоматически выводит типы без дублирования.

Код-дифф: Эволюция DI

Устаревший подход

```
export class ProductService {  
  constructor(  
    private http: HttpClient,  
    private store: Store,  
    private router: Router  
  ) {  
    super(http, store, router);  
  }  
}
```

Современный подход

```
export class ProductService {  
  private http = inject(HttpClient);  
  private store = inject(Store);  
  private router = inject(Router);  
}
```


Иерархия Injector'ов в эпоху Standalone

Platform Injector -> Null Injector

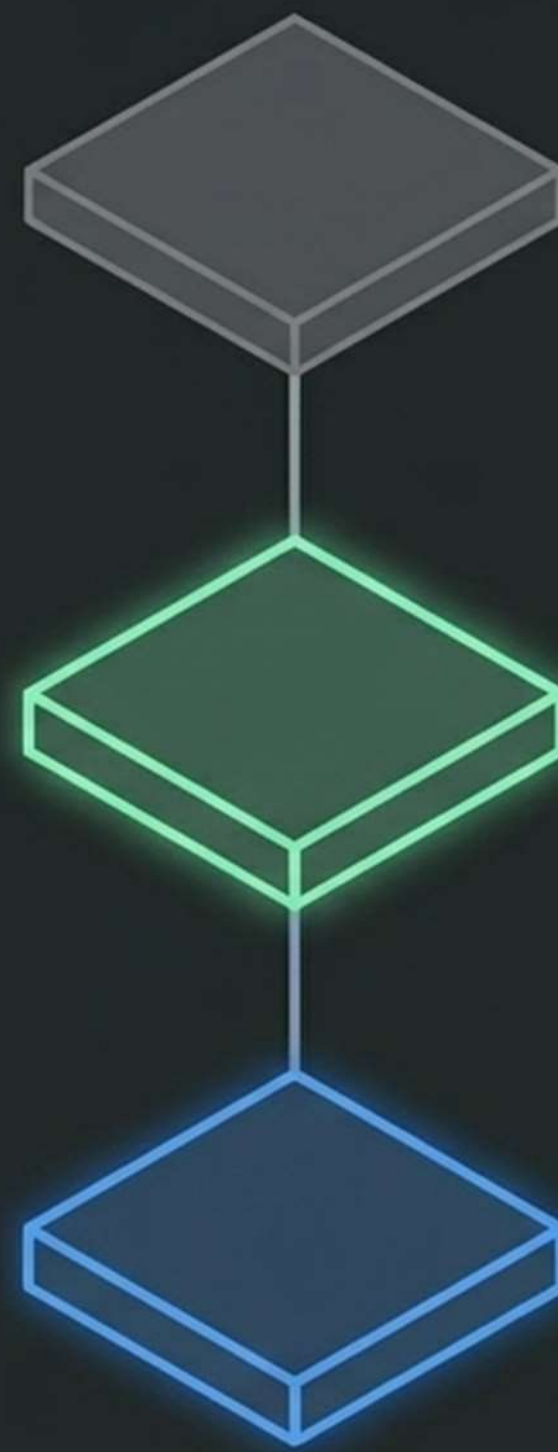
- Вершина дерева, выбрасывает `NullInjectorError`.

Environment Injector (Глобальный)

- Заменяет старый `Module Injector`.
- Управляет глобальными синглтонами (`providedIn:`
- Инициализируется через `bootstrapApplication(providers)`.

Element / Node Injector (Локальный)

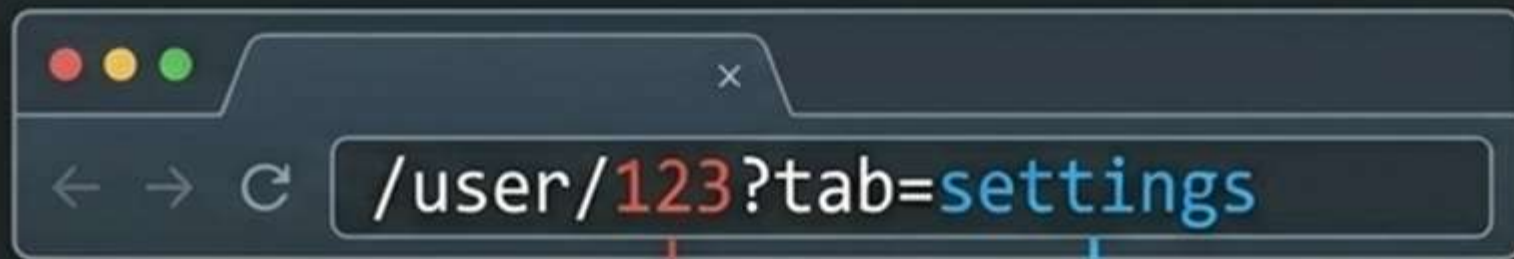
- Создается для каждого DOM-узла с компонентом/директивой.
- Определяется через `providers` внутри `@Component`.



Поиск
зависимости
(Lookup)

Маршрутизация следующего поколения: withComponentInputBinding

Замена `RouterModule.forRoot` на `provideRouter`. Автоматический маппинг данных роутера напрямую в свойства компонента.

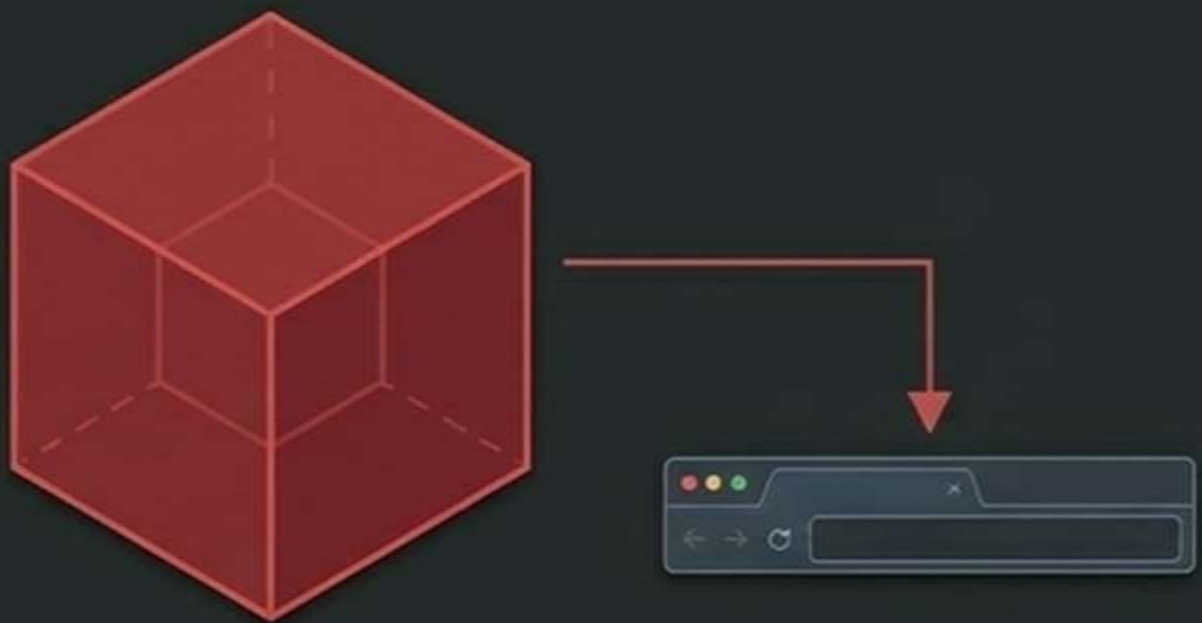


```
@Component({ ... })
export class UserComponent {
  → @Input() id!: string;
  → @Input() tab!: string;
}
```

- Полный отказ от инъекции `ActivatedRoute`.
- Устранение ручных подписок (subscriptions) на параметры.
- Компонент **отвязывается** от роутера (легче тестировать в изоляции).

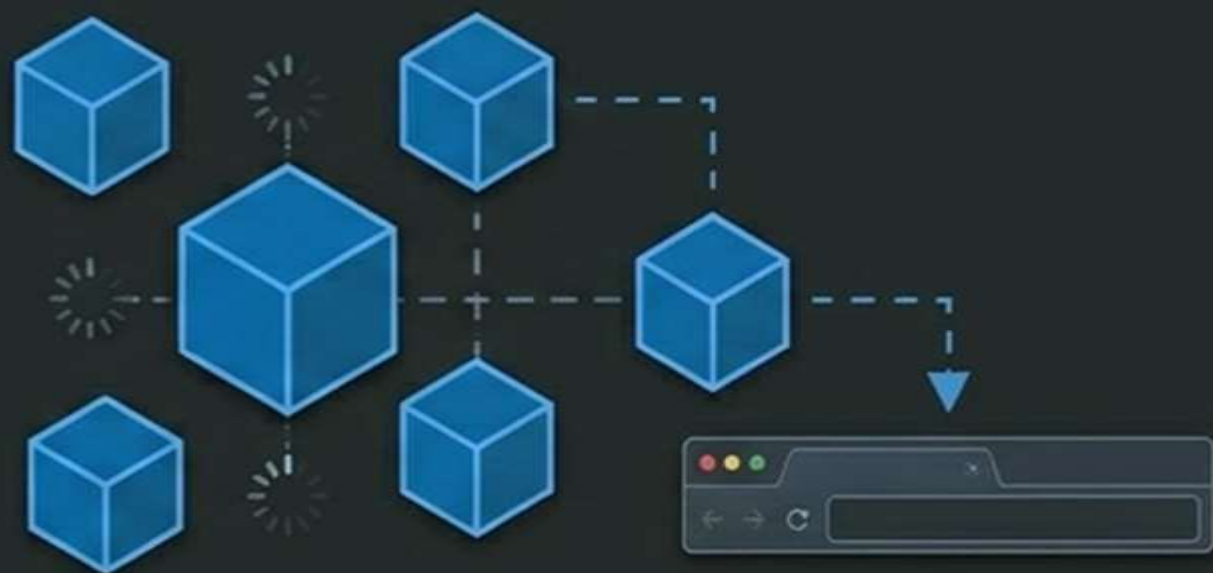
Стратегии загрузки: Баланс производительности

Eager Loading (Синхронная)



- Компоненты компилируются в **основной бандл**.
- Когда использовать: Только для **критического пути** (Главная страница, Shell приложения).

Lazy Loading (Асинхронная)



- Сплиттинг кода на отдельные **JS-чанки** (`loadComponent`, `loadChildren`).
- Загрузка происходит только при фактическом переходе на маршрут.
- Когда использовать: Для всех **фича-модулей**, **тяжелых диалогов** и **скрытых страниц**.

Tech Note: Router автоматически разворачивает default **экспорты динамических импортов**, устраняя необходимость в вызовах `.then()`.

Синтез: Идеальный компонент 2026 года

1. Standalone by Default: Нет NgModule, локальный скоуп импортов.

2. Signal-based Inputs: Использование input() вместо декораторов.

```
@Component({
  selector: 'user-profile',
  standalone: true,
  imports: [AvatarUI],
  template: `
    @if (user(); as u) {
      <avatar-ui [id]="u.id" />
    }
  `
})
export class UserProfile {
  → userId = input<string>();
  api = inject(UserService);
  user = httpResource(() => this.api.getUser(this.userId()));
}
```

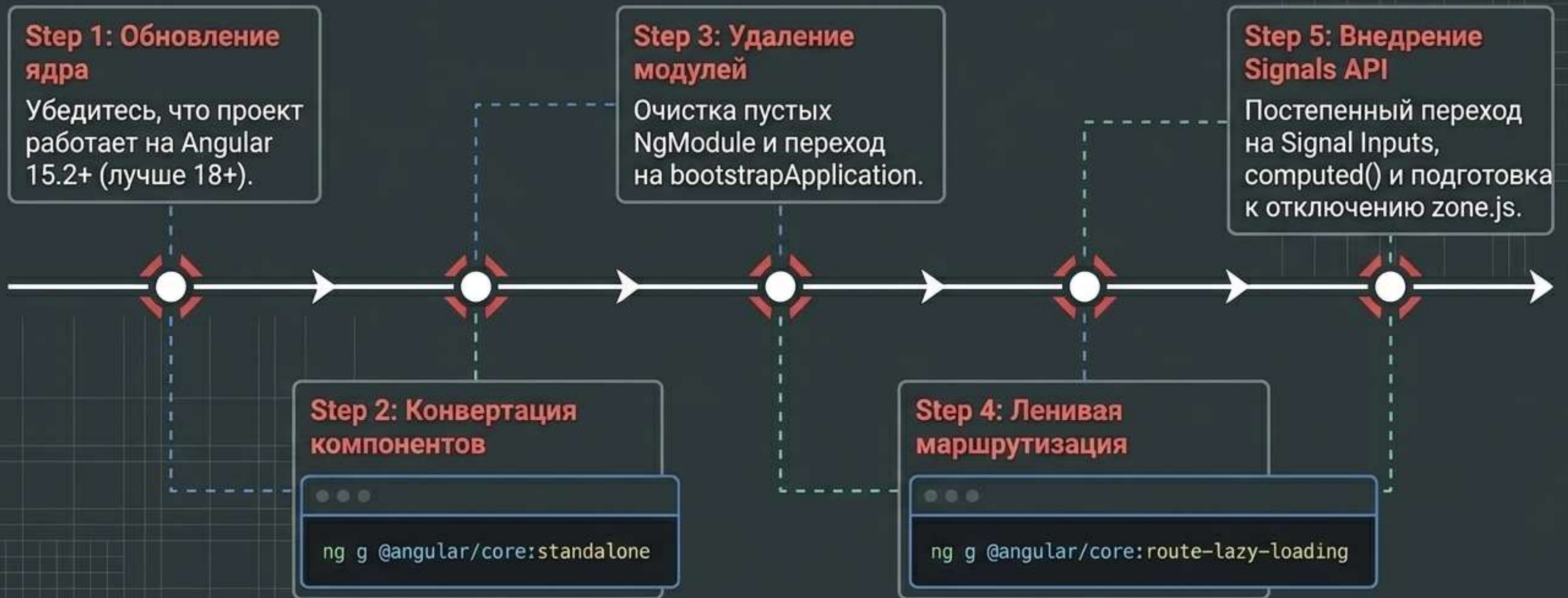
3. Modern Control Flow: @if и @for прямо в шаблоне.

4. Functional DI: Использование inject() вместо конструктора.

5. Modern Data Fetching: Использование httpResource() для чтения данных.

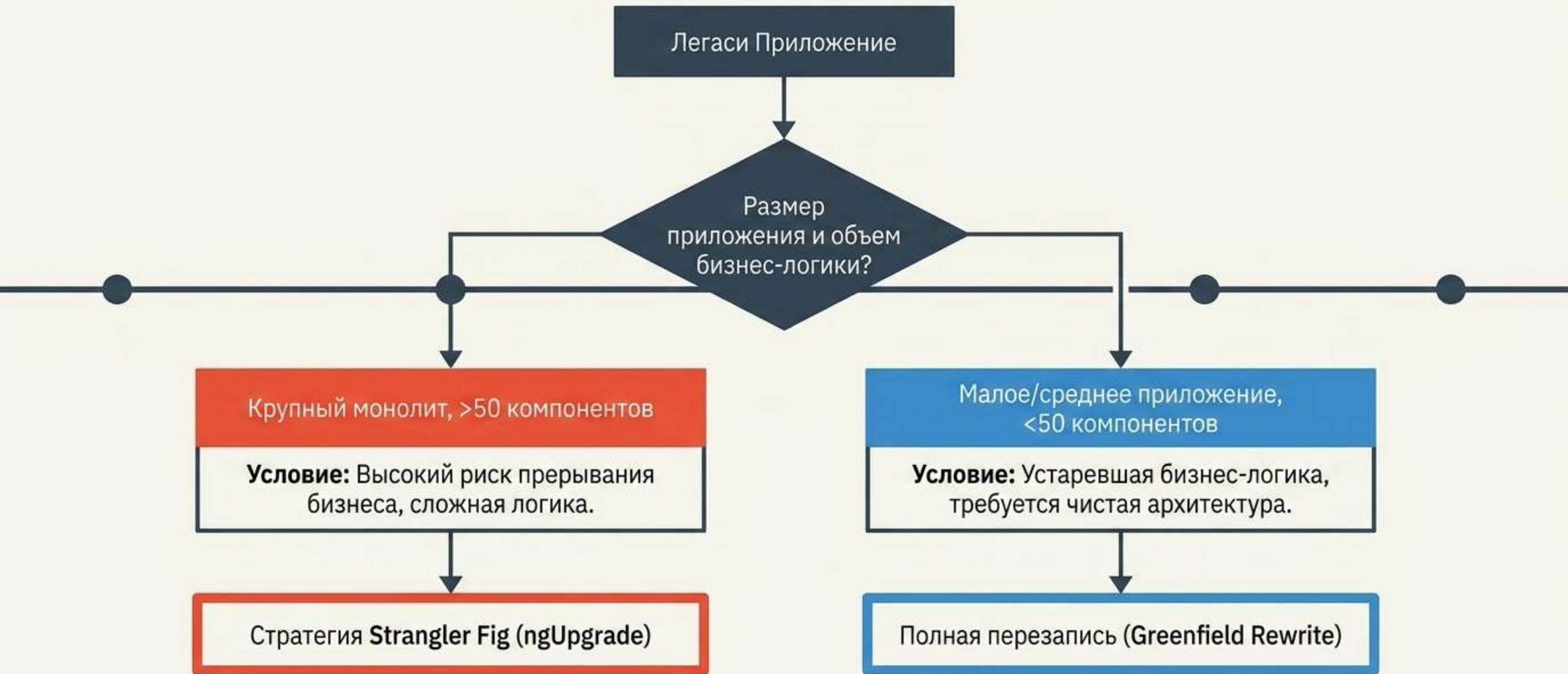
[Zoneless Change Detection Ready]

Дорожная карта миграции (Action Plan)

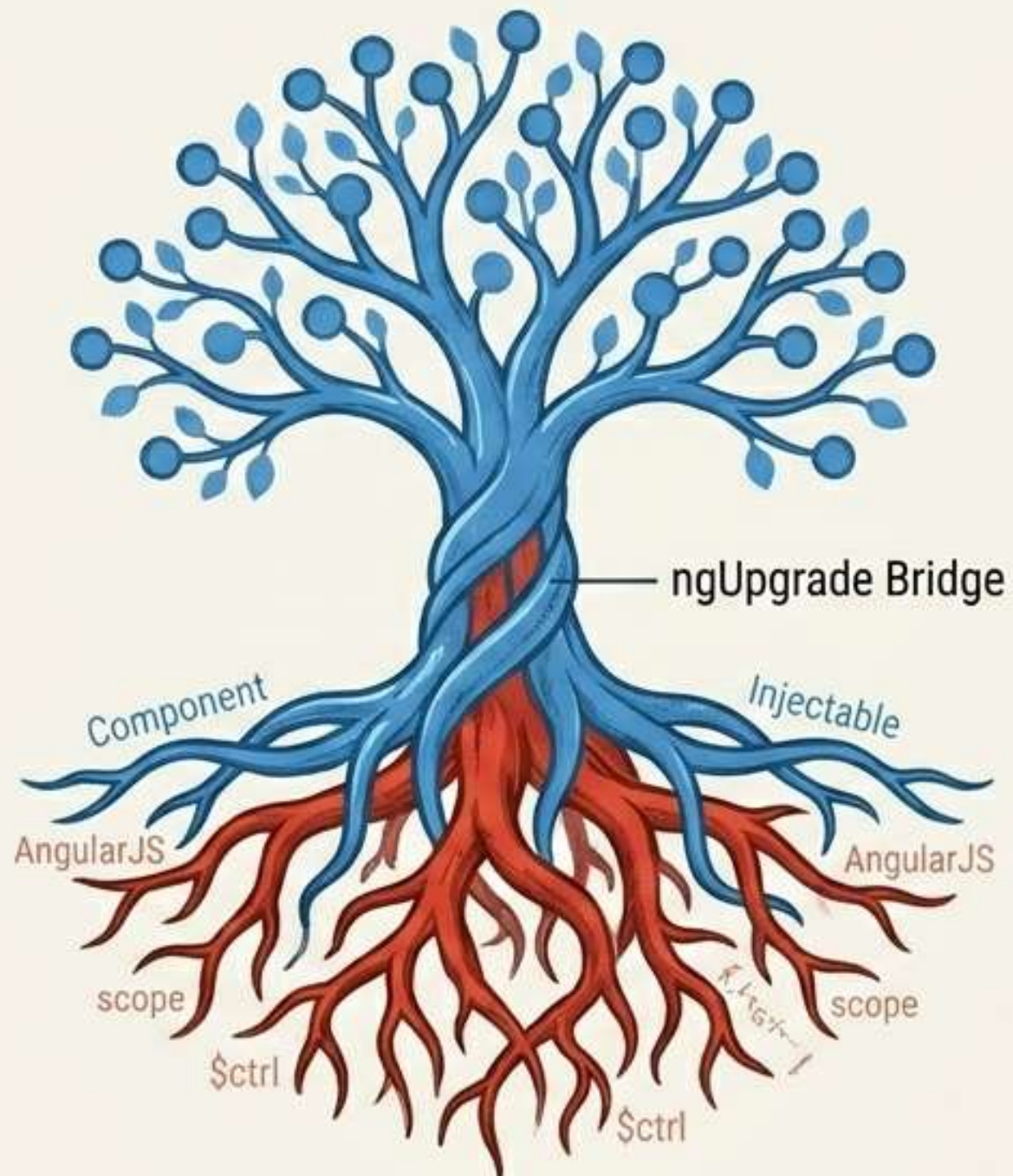


Миграция — это не "переписывание с нуля". Standalone и NgModules могут работать параллельно. Мигрируйте инкрементно.

Матрица принятия решений по миграции



Стратегия 1: Постепенная миграция (ngUpgrade)



Механика:

Запуск AngularJS и Angular параллельно в одном контексте. Компоненты взаимодействуют через специальный мост.

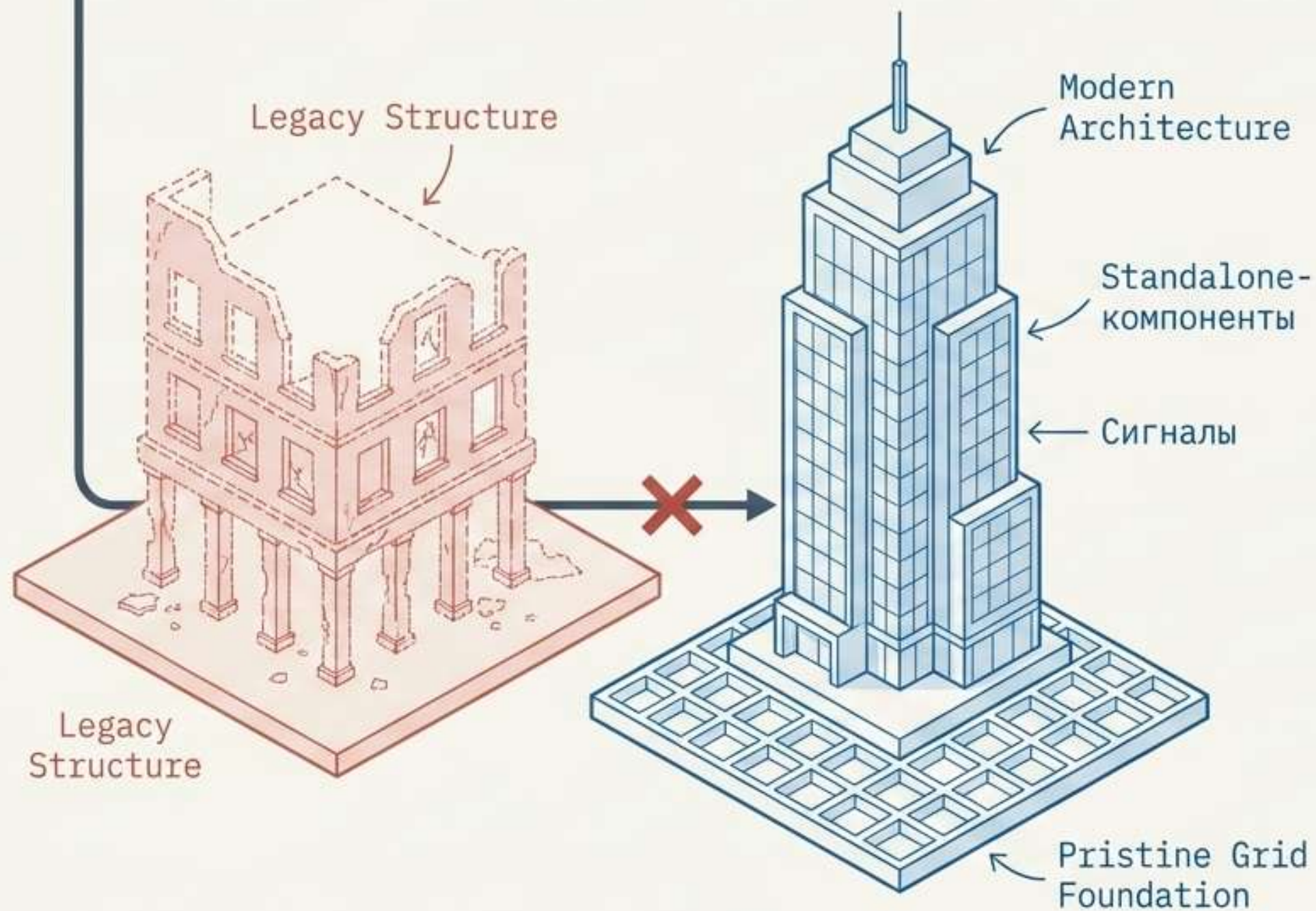
Плюсы:

- ✓ Инкрементальный подход, минимальный риск для непрерывности бизнеса.
- ✓ Распределение затрат на несколько циклов релизов.

Минусы:

- ✗ Значительное увеличение размера бандла в переходный период.
- ✗ Необходимость поддерживать две кодовые базы и обучать команду двум контекстам.

Стратегия 2: Полная перезапись (Greenfield Rewrite)



Механика:

Создание новой архитектуры с нуля с использованием Standalone-компонентов и Сигналов.

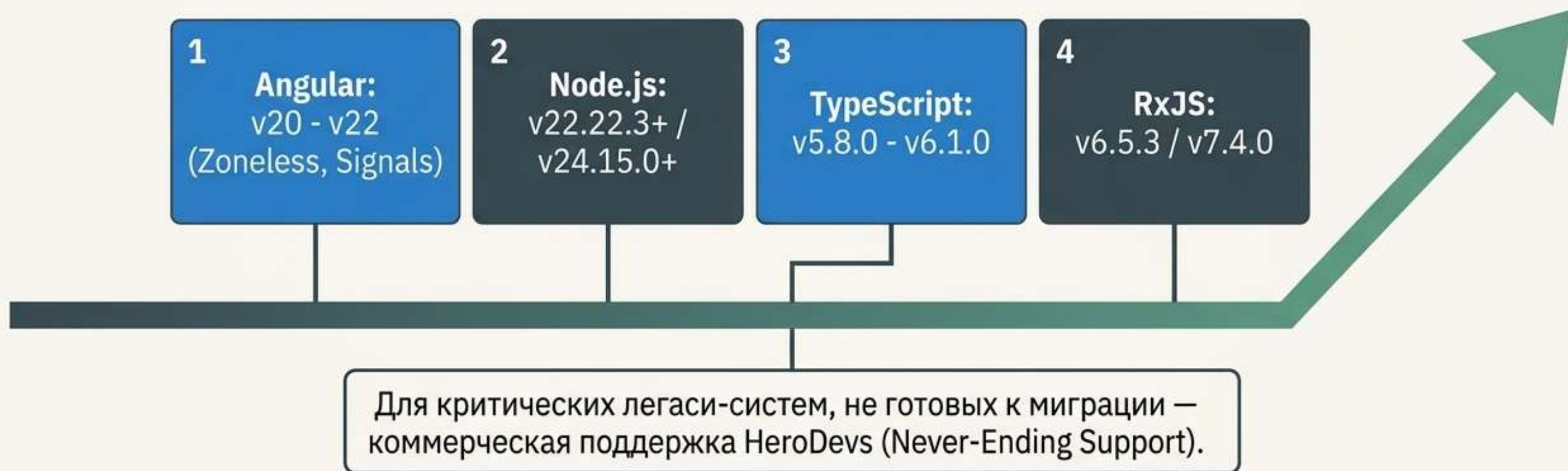
Плюсы:

- ✓ Полное избавление от накопленного технического долга.
- ✓ Идеально чистая архитектура и максимальная производительность.
- ✓ Относительно быстрый процесс для малых приложений (2-6 месяцев).

Минусы:

- ✗ Замораживание разработки новых фич в старом приложении на время перезаписи.

Целевой стек 2026 и Эпоха Возрождения



Angular больше не догоняет конкурентов — он устанавливает новый стандарт Enterprise-разработки. Отказ от легаси — это не просто обновление зависимостей, это инвестиция в производительность и безопасность бизнеса.